

설비 고장을 예측하는 AI 데이터 분석

NumPy 기초

배열 생성과 구조

목차

구성: 배열 생성 / 배열 구조와 속성

PART 01	배열 생성	NumPy 개념과 배열 만들기 · 개념·프레임	23분
PART 02	배열 구조와 속성	속성 확인과 형태 변환 · 개념·프레임	42분

NumPy를 배우는 이유

설비 센서 데이터는 하루에 수백만 개씩 쌓이는 숫자 묶음

NumPy는 이 숫자 묶음을 빠르게 계산하는 데이터 분석의 출발점

공장 설비 한 대에는 온도·진동·압력 센서가 붙어 1초에도 여러 번 값을 쏟아냅니다.

설비가 수십 대면 하루에 수백만 개의 숫자가 쌓이고, 사람이 직접 계산할 수는 없습니다.

그래서 컴퓨터에게 맡기는데, 그 가장 기본 도구가 바로 **NumPy**입니다.

앞으로 배울 **Pandas**나 **그래프 도구**도 모두 NumPy 위에서 돌아갑니다.

NumPy의 위치 — 데이터 분석 도구

NumPy는 모든 데이터 분석 도구의 기초

도구	역할	비고
NumPy	숫자 묶음 계산	가장 기초
Pandas	표 데이터 처리	NumPy 기반
Matplotlib	그래프 시각화	NumPy 기반

파이썬 리스트의 한계

리스트는 숫자에 곱셈하면 **계산이 아니라 반복**이 일어남 — 배열은 한 줄로 해결

리스트

숫자에 곱셈하면 계산이 아니라 반복이 일어남.
모든 값을 계산하려면 **반복문**이 필요함.

배열

목록 전체에 **한 번에 계산** 적용.
반복문 없이 한 줄로 처리.

리스트와 배열 — 무엇이 다른가

같은 일을 해도 처리 방식이 다름

항목	리스트	배열
전체 계산	반복문 필요	한 줄로
처리 속도	느림	빠름
통계 계산	번거로움	함수 하나

배열(ndarray)이란 무엇인가

배열은 **같은 자료형**의 값만 담는다 — 그래서 빠르다

배열은 **정해진 형태**를 가진다 — 한 줄 또는 표 모양

NumPy에서 데이터를 담는 그릇을 **배열**, 정확히는 **ndarray**라고 부릅니다.

배열은 숫자면 전부 숫자, **같은 종류만 담습니다**.

덕분에 메모리에 차곡차곡 정렬되어 계산이 빨라집니다.

계란판처럼 같은 크기 계란만 정해진 자리에 담겨, 빠르게 찾을 수 있습니다.

배열의 두 가지 핵심 특징

같은 자료형 + 정해진 형태 — 제약처럼 보이지만 속도의 비결

같은 자료형

숫자면 전부 숫자.

리스트와 달리 섞어 담지 못함.

대신 계산이 빠름.

정해진 형태

한 줄짜리 배열, 또는 행과 열을 가진 표 모양.

모양이 명확함.

NumPy 불러오기

코드 맨 위에서 NumPy를 **np**라는 별명으로 불러오기

CODE

```
import numpy as np  
  
print(np.array([1, 2, 3]))  
# 출력: [1 2 3]
```

import numpy는 불러온다는 뜻, **as np**는 앞으로 np라는 짧은 별명으로 부르겠다는 뜻.

이 별명은 전 세계 분석가가 똑같이 쓰는 약속 — 새 코드는 항상 이 줄부터 시작.

별명 붙이기는 공통 약속

데이터 분석 라이브러리는 **짧은 별명**을 붙이는 같은 약속을 따른다

NumPy

```
import numpy as np
```

로 불러옴.

Pandas

```
import pandas as pd
```

로 불러옴.

그래프

```
import matplotlib.pyplot as plt
```

로 불러옴.

np.array로 배열 만들기

리스트를 `np.array`에 넣으면 배열이 됨 — 묶음 계산을 한 줄로

CODE

```
temp = np.array([70.5, 69.8, 73.7])
print(temp)
# 출력: [70.5 69.8 73.7]
print(temp + 5)
# 출력: [75.5 74.8 78.7]
```

출력을 보면 리스트와 달리 **선표 없이 공백**으로 값이 구분됩니다.

`temp + 5`에서 모든 값에 한꺼번에 5가 더해지는 **묶음 계산** — 반복문이 전혀 없습니다.

배열 만들기 — 기억할 순서

값을 리스트로 묶고 `np.array`에 넣고 출력으로 확인

STEP
01

답을 값들을 리스트로 묶기 괄호 `[]`로 묶는 것이 첫걸음

STEP
02

`np.array` 괄호 안에 리스트 넣기 리스트 하나로 묶어 전달

STEP
03

출력으로 공백 구분 확인하기 공백 구분이면 배열이 제대로 만들어진 것

배열의 자료형은 자동으로 정해짐

모두 정수면 **정수 배열**, 하나라도 소수점 있으면 **실수 배열**

배열은 같은 자료형만 담으므로 더 넓은 쪽으로 자동 통일

np.array는 우리가 넣은 값을 보고 **자료형을 스스로 정합니다.**

모두 정수면 정수 배열이 되지만, **1, 2, 3.0**처럼 소수점이 하나라도 섞이면 전체가 실수 배열이 됩니다.

배열은 같은 자료형만 담을 수 있어서, 더 넓게 담을 수 있는 **실수 쪽으로 통일**합니다.

→ 만든 배열의 **dtype**이 예상과 다를 때 **이 규칙을 떠올리세요.**

실습 1. 센서값 배열 만들기

목표

섭씨 센서값 리스트를 배열로 만들고 화씨 단위로 한 번에 변환

단계

- 섭씨 측정값 리스트를 np.array로 배열 생성
- 배열에 곱셈과 덧셈을 묶은 연산으로 적용해 화씨로 변환
- 변환된 배열 출력

예상 결과

섭씨 네 개 값이 화씨 네 개 값으로 한 번에 바뀐 배열 출력

np.arange로 연속 정수 만들기

일정 간격의 **연속된 숫자**를 자동으로 생성

CODE

```
print(np.arange(5))  
# 출력: [0 1 2 3 4]  
print(np.arange(0, 10, 2))  
# 출력: [0 2 4 6 8]
```

np.arange(5)는 0부터 4까지 — 핵심은 **끝 숫자 5가 제외**된다는 점.

세 번째 값 **2**는 간격 — 0부터 2씩 건너뛰니다.

np.arange의 핵심 규칙

끝 숫자는 포함되지 않음 — arange(5)는 5가 아니라 4까지

파이썬 전체에서 통하는 "시작 포함, 끝 제외" 규칙

가장 자주 하는 실수: arange(5)를 0부터 5까지 여섯 개라고 착각.

실제로는 **0부터 4까지 다섯 개**입니다.

이건 np.arange만의 규칙이 아니라 **파이썬 전체의 약속**입니다.

곧 배열 **슬라이싱**에서도 똑같이 끝이 제외됩니다.

→ 소수점 간격은 미세한 오차가 생길 수 있으니 정확히 나눌 땐 **linspace** 사용.

np.linspace로 균등 분할하기

시작과 끝 사이를 정해진 개수로 똑같이 분할

CODE

```
print(np.linspace(0, 1, 5))  
# 출력: [0. 0.25 0.5 0.75 1. ]
```

세 번째 값 **5**는 간격이 아니라 **만들 점의 개수**.

arange는 간격을 정하지만 linspace는 개수를 정합니다.

→ 출력의 끝값 **1**이 포함되어 있음 (arange는 끝값 제외, linspace는 끝값 포함).

arange와 linspace — 무엇을 정하나

간격이 중요하다면 `arange`, 개수가 중요하다면 `linspace`

arange

간격을 정함.

개수는 계산해야 함.

끝값 제외.

linspace

개수를 정함.

간격은 자동 계산.

끝값 포함.

그래프 축을 만들 때는 보통 개수를 정하니 **linspace**를 자주 씁니다.

zeros · ones · full 초기화 배열

같은 값으로 가득 찬 배열을 한 번에 생성

CODE

```
print(np.zeros(3))  
# 출력: [0. 0. 0.]  
print(np.full(3, 7))  
# 출력: [7 7 7]
```

`np.zeros(3)`은 0이 세 개 든 배열. 출력의 **0.**은 실수 배열이라는 표시.

`np.full(3, 7)`은 7이 세 개. `np.ones`는 1로 채웁니다.

초기화 함수 — 무엇으로 채우나

용도에 따라 **0 · 1 · 특정 값**으로 채우는 세 가지 함수

np.zeros

0으로 채움.

결과를 담을 빈 그릇을 준비할 때.

np.ones

1로 채움.

비율이나 가중치 출발점으로.

np.full

특정 값으로 채움.

기준값을 설정할 때.

2차원은 형태를 괄호로 한 번 더 묶어 전달합니다.

배열 생성 함수 정리

만들고 싶은 배열에 따라 함수 선택

함수	언제	예시
np.array	값을 알 때	센서 측정값
np.arange	간격 필요	측정 시점
np.linspace	개수로 나눔	그래프 축
np.zeros	0으로 채움	빈 그릇

np.full은 표에 없지만 특정 값으로 채울 때 사용합니다.

함수 선택 — 스스로 질문하기

값을 아는가? **array** / 간격이 필요한가? **arange**

개수로 나누는가? **linspace** / 같은 값으로 채우는가? **zeros** · **full**

함수를 고르는 가장 쉬운 방법은 스스로 질문을 던지는 것입니다.

완벽히 외우지 못해도 됩니다. 실습을 반복하면 손이 먼저 기억합니다.

→ 중요한 건 내가 만들 배열이 어떤 종류인지 먼저 생각하고 함수를 고르는 습관.

실습 2. 균등 간격 배열 만들기

목표

지정한 구간을 정해진 개수로 균등 분할한 배열 생성

단계

- 시작값·끝값·개수를 정해 np.linspace로 구간을 균등 분할
- 만들어진 배열과 각 값의 간격 확인
- 결과 배열 출력

예상 결과

0부터 100까지를 다섯 개로 나눈 균등 간격 배열 출력

실습 3. 측정 시간축 배열 만들기

목표

측정 시점을 나타내는 시간축 배열을 일정 간격으로 생성

단계

- 시작값·끝값·간격을 정해 np.arange로 시점 배열 생성
- 간격을 바꿔가며 시점 개수 변화 관찰
- 시간축 배열 출력

예상 결과

0초부터 일정 간격으로 증가하는 측정 시점 배열 출력

1차원과 2차원 배열

배열은 모양에 따라 **1차원**과 **2차원**으로 나뉜다

1차원

값이 **한 줄로 늘어선 형태**.

센서 하나가 시간에 따라 측정한 값.

2차원

행과 열을 가진 표 모양.

가로줄이 행, 세로줄이 열.

설비 데이터를 본격 분석할 때는 대부분 **2차원**을 다루게 됩니다.

차원이란 값을 늘어놓는 방향의 수

1차원은 한 방향(한 줄), 2차원은 두 방향(가로·세로)

설비 데이터는 보통 **행=시점**, **열=센서**의 2차원 표

차원을 쉽게 풀면, **값을 늘어놓는 방향의 개수**입니다.

1차원은 한 줄로 선 사람들처럼 앞뒤 순서만 있고,

2차원은 극장 좌석처럼 몇 번째 줄 몇 번째 자리 두 정보로 위치가 정해집니다.

온도·진동·압력 센서가 여러 시점에 측정하면, **행이 시점이고 열이 센서**인 2차원 표가 됩니다.

→ 1차원은 그 표의 한 줄을 떼어낸 것.

배열의 차원(ndim)

ndim으로 배열이 몇 차원인지 확인 — 괄호 없는 속성

CODE

```
a = np.array([1, 2, 3])
b = np.array([[1, 2], [3, 4]])
print(a.ndim)
# 출력: 1
print(b.ndim)
# 출력: 2
```

ndim은 number of dimensions, 차원 개수.

한 줄짜리는 1, 표 모양은 2. ndim 뒤에 괄호가 없다는 것이 핵심.

속성과 함수의 차이

함수는 동작을 실행 — **괄호 있음** (예: `np.array()`)

속성은 정보를 꺼내봄 — **괄호 없음** (예: `배열.ndim`)

무언가를 시키는 **명령(함수)**은 괄호가 붙습니다. `np.array()`는 "배열을 만들어라"라는 동작.

배열이 이미 가진 정보를 꺼내 보는 **속성**은 괄호가 없습니다.

차원·형태·크기 같은 정보는 배열이 만들어질 때 **이미 달려 있어서**, 새로 계산하는 게 아니라 들여다보는 것.

→ 이 구분은 **shape, size, dtype**에도 똑같이 적용.

배열의 형태(shape)

shape으로 행과 열 크기 확인 — 앞=행, 뒤=열

CODE

```
b = np.array([[1, 2, 3], [4, 5, 6]])  
print(b.shape)  
# 출력: (2, 3)
```

2행 3열 배열의 shape이 괄호로 묶인 **(2, 3)**으로 나옵니다.

앞 숫자 **2가 행**, 뒤 숫자 **3이 열**. 이 순서는 2차원 인덱싱에서도 똑같이 쓰입니다.

괄호로 묶여 나오는 건 결과가 **튜플**이기 때문.

shape 읽는 법

분석가가 데이터를 처음 만나면 가장 먼저 던지는 질문 — **shape**이 뭐지?

(5,)

값 5개인 1차원 배열.

쉼표는 오타가 아니라 값 하나짜리 묶음이라는 표시.

(2, 3)

2행 3열.

앞이 행, 뒤가 열.

(1000, 20)

시점 1000개,
센서 20개.

배열의 크기(size)

size로 값의 전체 개수 확인 — 행 × 열

CODE

```
b = np.array([[1, 2, 3], [4, 5, 6]])  
print(b.size)  
# 출력: 6
```

2행 3열 배열의 size는 $2 \times 3 = 6$. 표 안의 칸 총 개수.

shape은 "어떤 모양인가", size는 "값이 모두 몇 개인가".

size도 속성이라 괄호가 없습니다.

shape과 size의 관계

shape은 모양, size는 전체 개수

shape의 숫자들을 곱하면 size — (500, 12)면 size는 6000

shape은 데이터가 어떤 모양으로 펼쳐졌는지,
size는 그 안에 값이 몇 개인지를 알려줍니다.

둘은 연결되어 있습니다 — **shape의 숫자들을 모두 곱하면 size.**

→ 모양은 바꿀 수 있어도, 담긴 값의 총 개수(size)는 변하지 않습니다.

→ 이걸 곧 배울 **reshape**에서 중요해집니다.

자료형(dtype)

dtype으로 값의 종류 확인 — 정수 int, 실수 float

CODE

```
a = np.array([1, 2, 3])
b = np.array([1.5, 2.7, 3.1])
print(a.dtype)
# 출력: int64
print(b.dtype)
# 출력: float64
```

정수는 int, 실수는 float. 뒤의 64는 저장 공간 크기.

dtype을 꼭 확인해야 하는 이유

데이터를 불러온 직후 **dtype 확인**이 계산 오류를 막아줌

정수 int

소수점 없는 값.

측정 순번, 설비 번호.

실수 float

소수점 있는 값.

온도, 진동.

숨은 글자

숫자처럼 보여도 글자로 저장되어 있을 수 있음.

계산 시 오류 발생.

형변환(astype)

astype으로 자료형 바꾸기 — 실수→정수는 소수점 버림

CODE

```
a = np.array([1.7, 2.3, 3.9])
print(a.astype(int))
# 출력: [1 2 3]
```

실수 배열에 **astype(int)**를 하면 정수로 바뀝니다.

꼭 짚어야 할 점 — **1.7이 2가 아니라 1**이 됩니다. 1.9도 1.

→ 반올림이 아니라 소수점 아래를 그냥 버립니다.

astype 사용 시 주의점

실수를 정수로 바꾸면 **소수점 아래를 버림** (1.9 → 1)

원본은 그대로, 새 배열이 생김 — **결과를 새 이름에 담기**

99.9를 정수로 바꾸면 100이 아니라 99가 됩니다.

정밀한 측정값은 함부로 정수로 바꾸지 않는 게 좋습니다.

astype는 변환된 **새 배열을 만들 뿐 원본은 그대로**입니다.

결과를 계속 쓰려면 `z = a.astype(int)`처럼 새 이름에 담아야 합니다.

→ 그냥 실행만 하고 안 담으면 **결과가 사라집니다.**

배열 속성 종합 정리

데이터를 만나면 가장 먼저 확인하는 네 가지

속성	알려주는 것	괄호
ndim	차원 개수	없음
shape	행·열 크기	없음
size	값 총 개수	없음
dtype	자료형	없음

shape의 숫자 개수가 ndim이고, shape을 곱하면 size. dtype만 모양과 무관한 독립 정보.

데이터 구조 파악 4단계

새 데이터를 만났을 때 따라야 할 순서 — 운전 전 거울·계기판 확인과 같음

STEP
01

ndim으로 차원 확인
차원? 2차원? 이후 작업 방향을 정함

STEP
02

shape으로 행·열 크기 파악
데이터 규모를 한눈에

STEP
03

size로 양, **dtype**으로 자료형 점검
계산 가능한 상태인지 확인

실습 4. 배열 구조 확인하기

목표

표 모양 배열의 차원·형태·개수를 속성으로 확인

단계

- 설비별 측정값을 담은 이차원 배열 준비
- ndim으로 차원, shape으로 형태, size로 전체 개수 확인
- 세 속성값 출력

예상 결과

차원 2, 형태 (2, 3), 개수 6 출력

실습 5. 자료형 확인과 변환하기

목표

배열의 자료형을 확인하고 정수형으로 변환

단계

- 소수점이 있는 측정값 배열 준비
- dtype으로 현재 자료형 확인
- astype으로 정수형으로 변환한 새 배열 출력

예상 결과

자료형 float64 확인, 소수점이 잘린 정수 배열 출력

reshape로 형태 바꾸기

값은 그대로, 모양만 바꾸기 — 값 개수는 같아야 함

CODE

```
a = np.arange(12)
print(a.reshape(3, 4))
# 출력: 3행 4열 배열 (0~11)
print(a.reshape(-1, 2))
# 출력: 6행 2열 (행은 자동 계산)
```

값 12개짜리를 **3행 4열**로 바꿈. 값은 그대로고 모양만 변함.

1을 쓰면 NumPy가 나머지 크기를 자동 계산.

reshape의 규칙

바꾸기 전후 **값 개수가 같아야** 함 — 12개를 3행 5열로는 불가

행이나 열에 **-1**을 넣으면 나머지를 자동 계산

12개는 3행 4열이나 2행 6열은 되지만, 3행 5열은 15칸이라 안 됩니다.

cannot reshape 오류는 거의 이 문제.

데이터가 많아 개수를 세기 어려울 때, **열만 정하고 행은 -1**로 두면 NumPy가 맞춤.

→ 단, -1은 **행과 열 중 한 곳에만** 쓸 수 있습니다.

flatten으로 1차원 만들기

표 모양을 **한 줄로 펴기** — 위 행부터 왼쪽에서 오른쪽으로

CODE

```
b = np.array([[1, 2, 3], [4, 5, 6]])  
print(b.flatten())  
# 출력: [1 2 3 4 5 6]
```

reshape이 한 줄을 표로 만든다면, **flatten**은 반대로 표를 **한 줄로 펴**니다.

퍼지는 순서: **위 행부터, 그 안에서 왼쪽에서 오른쪽** — 책 읽는 방향.

첫 행 1, 2, 3을 다 펴고 둘째 행 4, 5, 6을 이어 붙입니다.

flatten의 특징

위 행부터 **왼쪽에서 오른쪽** 순서로 펴 (책 읽는 방향)

표 전체에서 통계 낼 때, **전체를 한꺼번에 다룰 때** 유용

이 순서를 알면 **결과를 미리 예상**할 수 있고, 퍼진 값이 원래 어느 위치에서 왔는지 추적할 수도 있습니다.

여러 센서값이 표로 있는데 **센서 구분 없이 전체 평균**을 구하고 싶을 때,
flatten으로 한 줄로 편 뒤 계산하면 됩니다.

→ flatten도 **원본을 바꾸지 않고 새 배열을 만듭니다.**

형태 변환 정리

모양을 바꾸는 두 함수 — 값은 그대로, 배치만 변경

함수	하는 일	결과
reshape	모양 바꾸기	원하는 행·열
flatten	한 줄로 펴기	1차원 배열

공통점 — 값 자체는 바꾸지 않고 **배치만 바꾸며, 값의 총 개수는 변하지 않음**.

한 줄로 들어온 데이터를 표로 정리할 땐 **reshape**, 표를 한 줄로 펴서 전체를 다룰 땐 **flatten**.

→ 둘 다 결과를 새 이름에 담아야 쓸 수 있습니다.

배열 다루기의 전체 흐름

데이터를 파일에서 불러오면 처음엔 원하는 모양이 아닐 때가 많음 — **만들기·확인·정리**가 하나의 흐름

STEP
01

만들기 np.array로 배열 생성

STEP
02

확인하기 속성으로 구조 파악 (ndim · shape · size · dtype)

STEP
03

모양 바꾸기 reshape으로 분석하기 좋은 모양으로 정리

실습 6. 배열 모양 바꾸기

목표

한 줄 배열을 값 개수에 맞는 표 모양으로 변환

단계

- 연속 정수 배열을 arange로 생성
- 값 개수에 맞는 행·열을 정해 reshape로 형태 변환
- 바뀐 배열 출력

예상 결과

여덟 개 값이 2행 4열 표 모양으로 바뀐 배열 출력

실습 7. 센서 데이터 표로 정리하기

목표

한 줄로 이어진 측정값을 행=시점, 열=센서 표로 정리

단계

- 시점과 센서 수를 곱한 개수만큼 연속값을 arange로 생성
- 행을 시점, 열을 센서 수로 정해 reshape로 표 형태 변환
- 정리된 표 배열 출력

예상 결과

여섯 개 값이 3행 2열(시점 3 × 센서 2) 표로 정리된 배열 출력

실습 8. 배열 생성부터 정리까지

목표

배열 생성, 구조 확인, 형태 변환을 하나의 흐름으로 수행

단계

- 센서 측정값을 np.array로 배열 생성
- shape과 dtype으로 구조 확인
- reshape으로 분석용 표 형태로 정리한 뒤 출력

예상 결과

형태와 자료형 확인 후 3행 2열 표로 정리된 배열 출력

데이터 분석의 기본 흐름

분석은 항상 만들기 → 구조 확인 → 모양 정리 순서

이 흐름이 몸에 배면 어떤 데이터도 다룰 수 있음

STEP 01

만들기

np.array

STEP 02

구조 확인

ndim · shape · size · dtype

STEP 03

모양 정리

reshape · flatten